

Capstone Checkpoint 3.1: RAG and Retrieval Design Integration

Author: Jacques Troussard

1. Retrieval Requirement and System Use Cases

Decision: Yes, a semantic retrieval mechanism is required.

Reasoning:

The primary bottleneck in our automated pipeline is when a document has messy, faded text, or conflicting entity resolution, causing the LLM to return low-confidence metadata. The agent must leave these fields blank and route the document to the manual review bucket.

When a human manually corrects and categorizes these failed documents in the UI, we save their corrections. By integrating a semantic retrieval layer, the agent can search this database of past human corrections when it processes a new, similar document. It pulls up the exact correction the human made and uses it as an example in the prompt, turning a repeat failure into a successful automated run.

2. Integration of Semantic Retrieval with External Data Source

External Data Source: Yes, the manual review results database is our external data source. It stores the historical "ground-truth" metadata corrections provided by the user.

Integration Mechanics:

1. Data Store: We run a lightweight, local vector index alongside our main SQLite database.

2. The Ingestion Pipeline: Whenever a user completes a manual review correction in the UI, the system generates a Structured Correction Payload. This payload pairs the finalized, validated Document Metadata JSON (the ground-truth fields corrected by the human) with a composite text representation of the document.

To construct this text representation, we extract a composite slice consisting of the first 1,000 characters and the last 1,000 characters of the OCR stream. This composite string is embedded using a lightweight, offline local model and saved to our local vector index, while the matching structured JSON is committed directly to our relational SQLite database.

3. The Agentic Tool: We add a new tool to our registry: `retrieve_past_corrections(ocr_text: str, k: int = 1)`. When the ReAct loop encounters a document where classification confidence is low, it invokes this tool to fetch the closest match from past manual reviews.

3. Demonstration of Retrieval-Grounded Output

Our system enforces a strict "no-guessing" policy to protect filesystem integrity: if the agent cannot confidently identify metadata fields, it leaves them blank and flags the file for human review. Semantic retrieval resolves this without breaking the policy.

Scenario: Processing a scanned document with unresolved document template fields.

Without Retrieval:

The OCR tool extracts fragmented text: "PWSA ACCT: 9912-X... DUE: \$34.50". The LLM's static training data doesn't know what "PWSA" is, and the invoice format is non-standard. Because it cannot confidently resolve the entity name or directory path, the agent leaves the metadata fields blank, halts processing, and writes the file to the needs_review directory.

- **Automation Rate:** 0% (Requires manual intervention).

With Retrieval:

The agent extracts the same text, recognizes low classification confidence, and calls `retrieve_past_corrections("PWSA ACCT: 9912-X")`. The vector index returns a matching record from three months ago where a human corrected a similar document:

- **Retrieved Context:** "For text containing 'PWSA ACCT: 9912-X', the user corrected the Entity to 'Pittsburgh Water & Sewer Authority' and the Destination Path to 'Personal/Utilities/Water/Pittsburgh_Water_and_Sewer_Authority'."

Grounded by this retrieved context, the LLM confidently and accurately fills out the metadata, constructs the path, and archives the file.

- **Automation Rate:** 100% (Fully automated based on past learning).

4. Key Retrieval Design Choices

Data Source Selection: A local, offline vector index storing embeddings of human-corrected OCR text snippets paired with their finalized metadata JSON. We do not index raw, successful, digital PDF text, as digital files already parse with high confidence. We only index the "hard cases" resolved by human review.

Document Segmentation and Chunking:

We do not chunk documents by arbitrary character counts (e.g., 500-token blocks), which destroys document context.

Instead, we chunk at the document boundary. Since physical mail is typically 1–3 pages, we embed only the first & last 1,000 characters of the OCR text (where the vendor name, account numbers, and dates reside). This keeps the vector store highly focused on identity markers and keeps our token usage low.

Retrieve Count (k): We retrieve exactly $k = 1$ result. Because we are looking for highly specific, repeated billing or tax documents, we only want the single closest match. This keeps our ReAct prompt clean, fast, and well within our local token budget.

5. Retrieval-Related Failure Mode and Mitigation

Potential Failure Mode & Retrieval Drift (False Positive Matches):

Because different utility bills look highly similar, the vector index might return a past correction for a "Duquesne Light" power bill when the agent is actually processing a "PGW" gas bill, simply because both share 90% of their layout text. If the agent blindly trusts and injects this context, it will misfile the gas bill in the electricity folder.

System Mitigation:

We implement a strict Anchor Key Validation Rule in Python. Before the retrieved context is sent to the LLM:

1. The system extracts unique alphanumeric anchors (such as account numbers or specific tax IDs) from both the current document OCR and the retrieved correction.
2. If the retrieved result has a different account number or vendor identifier than the active document, the system ignores the retrieved context and forces the document to the manual review queue. This prevents "retrieval drift" from causing silent misfiling.